

Automated Credit Approval System: A Comparative Study of Logistic Regression, XGBoost and K-Means

line 1: Jie Ma
line 2: MDTI
line 3: University of Ottawa
line 4: Ottawa, Canada
line 5: jma091@uottawa.ca

line 1: Kathy Tao
line 2: MDTI
line 3: University of Ottawa
line 4: Ottawa, Canada
line 5: ytao048@uottawa.ca

line 1: Ruijie Duan
line 2: MDTI
line 3: University of Ottawa
line 4: Ottawa, Canada
line 5: rduan027@uottawa.ca

line 1: Zhiyuan Ni
line 2: MDTI
line 3: University of Ottawa
line 4: Ottawa, Canada
line 5: zni052@uottawa.ca

line 1: Jia Sheng
line 2: MSysScEng
line 3: University of Ottawa
line 4: Ottawa, Canada
line 5: jshen025@uottawa.ca

line 1: Zhen Ye
line 2: MDTI
line 3: University of Ottawa
line 4: Ottawa, Canada
line 5: zye038@uottawa.ca

Abstract—This project uses the credit approval dataset to build a credit approval forecasting system. We first do the data cleaning, including missing values, category transformation, reducing outliers, and preprocessing for all features to build a model. After preprocessing, we use different methods such as logistic regression, XGBOOST, and K-means clustering. The logistic regression performs as well as a baseline model, while the XGBoost provides optimized performance as it has higher precision and the ability to decide decline or approval. K-means cluster also demonstrates the natural grouping, helping us to know the patterns of different kinds of applicants. However, the project built a full and repetitive workflow to improve the accuracy and consistency of credit approval

I. PROBLEM FRAMING

The Credit Approval is a key decision process of a financial agency since it decides if the applicants are eligible for loans, credit cards, and other financial products. Traditionally, this decision strongly relies on manually crafted rules, rigid barriers, and human judgment. Although this system, which was based on the rule, is easy, it cannot capture complex interactions among demography, behavior, and financial attributes. Thus, the finance agency could approve the client who is high risk or decline customers who have high credit, leading to income loss, low business efficiency, and opportunity loss.

The dataset from this project is from real credit filter scenarios, including anonymous applicants' information such as age, financial ratio, spending pattern, and demography. The history credit approval is a real label. The original credit data usually has missing values, a non-standard format, data distribution skewness, and potential inconsistency. All these problems make the risk model more complex and force the technician to use a more complex analysis technique.

Based on these challenges, the project aims to construct a reliable and explanatory machine learning system to forecast whether a new credit application should be approved. The aim is not only to try to replicate historical decisions but also to extract the deep pattern that is ignored by the traditional filtering mode.

This problem introduces three key research questions: 1. What is the relationship between applicants' numerical and categoric attributes and credit approval? What preprocessing

steps should we do to prepare for model building? 2. Which machine learning model, such as logical regression, XGBoost, could most effectively capture the potential mode inside the data? What is their performance on the evaluation metric, such as accuracy, precision, recall, F1 score, and ROC curve? 3 Can the unsupervised method, such as k-means, reveal the grouping that is aligned with risk level and provide extra explainability over the supervised method?

By solving these problems, the project aims to design a data-driven workflow to enhance the decision-making capability of the financial agency and lower the risk from human judgment. The final system could potentially make the credit filtering more expandable, fairer, and suitable for different applicants' situations.

II. DATA DESCRIPTION PREPROCESSING EXPLANATION

A. Dataset Overview

This project uses the UIC credit approval dataset as a data source. This dataset is a binary classification dataset that was widely adopted for financial risk evaluation. It includes 690 instances, which represent individual credit applications; each instance consists of 15 anonymous attributes and one binary decision description, which indicates if the applicant gets the approval ("+") or is denied ("-"). This attribute includes datatypes like category, numeric, and sequence. It represents the individual demographic features, financial indicators, and relevant information of applicants. The original datasets use placeholders ("?.") to represent the missing value. Thus, we should rebuild the useful feature space by systematically cleaning the data, category transformation, and descriptive evaluation in the following steps.

As Figure 1 shows, the A2, A3, A8, A11, A14, and A15 are continuous variables, while other attributes are categoric at 15 input variables. This dataset includes different data types, and data distribution is unbalanced, so it is great for a preprocessing strategy and model evaluation under heterogeneous conditions. Before applying any machine learning algorithm, our team does a comprehensive exploration to describe distribution characteristics, detect outliers, evaluate missing patterns, and understand the preliminary relationship among variables.

FIG. 1. Dataset Overview

Data columns (total 16 columns):				
#	Column	Non-Null Count	Dtype	
0	A1	678 non-null	object	
1	A2	678 non-null	float64	
2	A3	690 non-null	float64	
3	A4	684 non-null	object	
4	A5	684 non-null	object	
5	A6	681 non-null	object	
6	A7	681 non-null	object	
7	A8	690 non-null	float64	
8	A9	690 non-null	object	
9	A10	690 non-null	object	
10	A11	690 non-null	int64	
11	A12	690 non-null	object	
12	A13	690 non-null	object	
13	A14	677 non-null	float64	
14	A15	690 non-null	int64	
15	Class	690 non-null	object	
dtypes: float64(4), int64(2), object(10)				

B. Descriptive Statistics, Missing Values, and Class Distribution

We use the descriptive statistical method to summarize the numerical attributes (A2, A3, A8, A11, A14, A15) to recognize the distribution features and potential preprocessing needs. As Figure 2 shows, all numerical features exhibit a right-skewed distribution, and there are large differences between the median and maximum value. This could show that the statistical feature value of some applicants' finance and demography is high, which is normal, in the credit approval datasets. There are large value differences in the standard deviations of variable A3, A11, and A15, so these attributes have obvious differences. For example, the value of A15 is 0 to 100000 and has a heavy-tailed distribution. If there is no appropriate dealing, it could affect the stability of the model. In comparison, the interquartile range shows that the value of most applicants falls in a small interval, such as A2 for 22.6 to 38.2. This means outliers only take a small portion of the whole data.

FIG. 2. Descriptive statistics

Descriptive Statistics for Numeric Columns							
	A2	A3	A8	A11	A14	A15	
count	678.000000	690.000000	690.000000	690.000000	677.000000	690.000000	
mean	31.568171	4.758725	2.223486	2.408000	184.014771	1017.385387	
std	11.957862	4.978163	3.346513	4.86294	173.806768	5210.102390	
min	13.750000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	22.682500	1.000000	0.165000	0.000000	75.000000	0.000000	
50%	28.460000	2.750000	1.000000	0.000000	160.000000	5.000000	
75%	38.230000	7.207500	2.625000	3.000000	276.000000	395.500000	
max	80.250000	28.000000	28.500000	67.000000	2000.000000	100000.000000	

The datasets use the "?" symbol to compile a missing value. For the correct detection, we transform these to NAN. The missing pattern is sparse and mainly limited to categorical variables like A1, A2, A4, A5, A6, and A14. These attributes comprise 6 to 13 missing values, and all numerical attributes are complete. Since the missing values take a small portion and do not centralize any key variables, the simple imputation technique is enough for the completeness and consistency of the dataset, instead of sequential or iterative imputation. We use the median for numerical values and the mode for categorical features.

FIG. 3. Missing Value

Missing Values per Column	
A1	12
A2	12
A3	0
A4	6
A5	6
A6	9
A7	9
A8	0
A9	0
A10	0
A11	0
A12	0
A13	0
A14	13
A15	0
Class	0
dtype: int64	

As figure 4 shows the category of distribution, it is easy to build a model. The dataset comprises 55.5 "-" decision making and 44.5 "+" decision making, which results in relatively balanced results. This reduces the need for sampling techniques such as SMOTE and weighted sampling. Overall, this dataset has small and limited missing values, controllable data distribution, and balanced categories, providing a solid base for following pre-processing and forecasting model building.

FIG. 4. Class Distribution

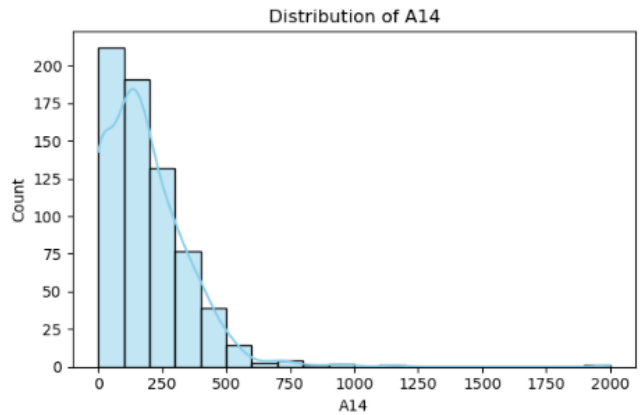
Class Distribution	
Class	
-	0.555072
+	0.444928
Name: proportion, dtype: float64	

C. Descriptive Statistics, Missing Values, and Class Distribution

To have a better understanding of attribution structure and the behavior of credit approval, we are doing a series of key point explorative analyses. Although there are many diagrams during the analysis procedure, we only include partial representative diagrams due to page limitations, such as figures 5,6,7,8 As a comprehensive visualization result, the analysis (whether shown or not) represents the following conclusion.

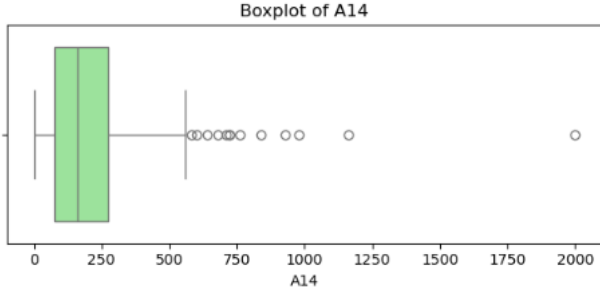
The distribution of the numerical map shows that all continuous variables are obviously right-skewed, most observed values fall in the low value area, which aligns with our conclusion from descriptive statistics.

FIG. 5. A14 Distribution



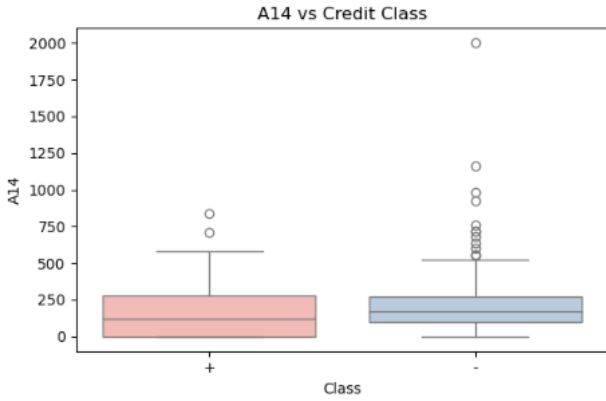
The Representative boxplot shows many high-value outliers, especially in variables such as A3, A11, A14, and A15. These outliers are not isolated, but they reflect the applicant's real situation and meaningful discrepancies. The stretching of the upper quartile also shows that the variability in the upper range is greater than the lower range. Thus, methods like percentile capping or robust modeling could reduce the disproportional effect of extreme values, but also can keep the information value.

FIG. 6. A14 Distribution



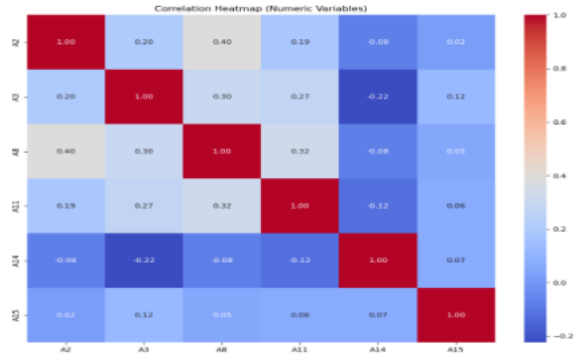
The class-conditional visualization shows that the approval applicants and decline applicants not only have differences in the central tendency but also have discrepancies in the distribution. For the attributes such as A3 and A8, the distribution of decline is wider, representing that variability could include the forecasting signal. These observational results support the non-linear model since it can capture the minor differences that can be ignored by the linear classification.

FIG. 7. A14 Class-conditioned visualizations



The correlation heatmap indicates that the correlation among the numerical attributes is weak to moderate. The non-existent multicollinearity proves that each feature provides unique information for classification. This uniqueness is beneficial for the method based on trees since it could form a stronger decision boundary by relying on diverse features.

FIG. 8. A14 Correlation Heatmap



Overall, the visualization diagnosis aligns with the conclusion above and represents that the dataset has the data distribution of skewness, obvious outliers, weaker feature relevance, and fewer missing values. These conclusions, based on visualization, provide a strong foundation for the following modeling build.

D. Data Preprocessing Overview

Machine learning models are highly dependent on the quality and consistency of input data. In practical financial settings such as banking, insurance, and credit assessment, raw datasets often contain inconsistencies that can significantly degrade model performance if mishandled. Credit approval datasets exemplify these challenges, featuring mixed data types, missing values, text codes, outliers, and symbolic values. Consequently, preprocessing is not a trivial preliminary step but a critical component of the entire modeling process.

This section proposes a systematic preprocessing workflow for the credit approval dataset. This dataset is one of the most used benchmarks for studying credit scoring and binary classification problems. Real-world financial datasets typically contain missing values, fuzzy categorical variables, extreme outliers, and structural noise, all of which negatively impact model performance. The objective of this section is to transform the raw dataset into a format suitable for machine learning while satisfying constraints such as standardization. The proposed workflow includes missing value imputation, type correction, percentile-based outlier control, deterministic label encoding, and creation of binary target variables. Additionally, this section provides multiple visualizations, including missing value plots, distribution comparison charts, and category balance diagrams, to demonstrate the impact of each step. The final processed dataset is fully reproducible and optimized for downstream classification models.

E. Dataset Description

The credit approval dataset contains 690 records, each representing a credit application and marked as approved (“+”) or rejected (“-”). This dataset includes 15 attributes, comprising:

- Numeric fields (A2, A3, A8, A11, A15)
- Categorical fields with binary values (A9, A10, A12)
- Multi-class nominal features (A1, A4, A5, A6, A7, A13, A14)

Missing values are denoted by the symbol “?”. Many categorical attributes consist of single-character alphanumeric codes whose meanings are not explicitly documented but must be treated consistently. Preliminary statistical analysis indicates that some attributes exhibit skewed distributions, while others contain outliers that may distort quantitative models. Based on these observations, we employed median imputation and percentile-based outlier capping methods.

FIG. 1. Missing Values per Attribute



Figure 1 presents the missing value count per attribute, confirming that the dataset requires substantial cleaning before modeling.

F. Methodology

a) Handling Missing Values

As shown in Figure 1, this illustrates the distribution of missing values within the dataset. Attributes A1, A2, and A14 exhibit the highest incidence of missing values, while attributes A4 through A7 show moderate rates of missingness. These patterns indicate that appropriate imputation strategies must be employed during model training to prevent bias. Numeric attributes are imputed using the median rather than the mean, as the median reduces the impact of outliers and provides greater stability in skewed financial data distributions. Categorical attributes are imputed using the mode to preserve the most frequently occurring category and ensure deterministic coding.

b) Numeric Conversion and Outlier Mitigation

FIG. 2. A2 Distribution Before Capping

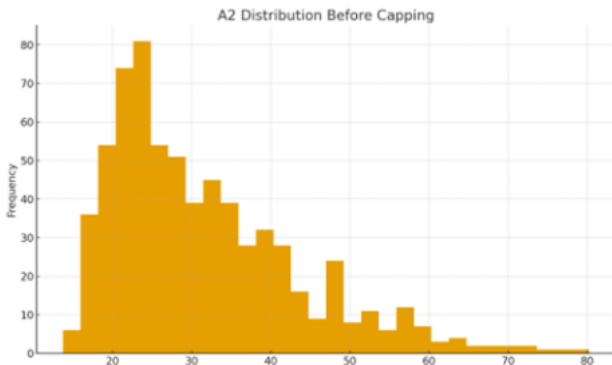


Fig. 2 illustrates the distribution of attribute A2 before applying any capping or outlier handling technique. The histogram shows a long right tail, indicating the presence of extreme values and a skewed distribution. Such outliers can negatively influence statistical assumptions and model stability, particularly distance-based or linear algorithms. Therefore, preprocessing steps such as capping are necessary to reduce the impact of extreme observations.

FIG. 3. A2 Distribution After Capping

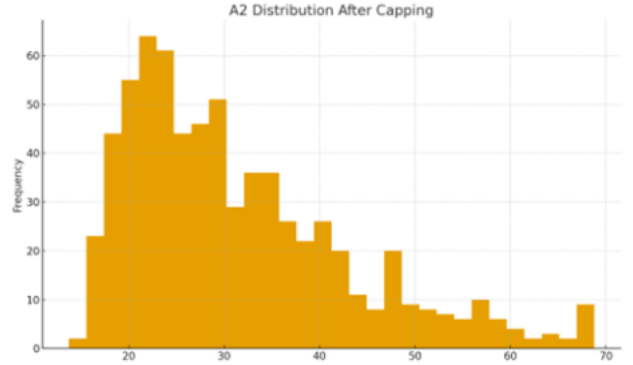


Fig. 3 presents the distribution of attribute A2 after applying a 99th-percentile capping strategy. Compared with the pre-capping distribution, the long right tail is effectively reduced while the central mass of the data remains preserved. This indicates that the capping procedure successfully mitigates the influence of extreme outliers without distorting the original shape of the distribution. Such controlled smoothing improves numerical stability for downstream machine-learning models.

c) Deterministic Categorical Encoding

Initially, some columns stored numerical information as text. Using pandas to numeric () function, we explicitly convert these columns to a numerical format through type coercion. All invalid entries were converted to NaN values and subsequently imputed. Outliers were capped at the 99th percentile. This approach ensures extreme financial values, such as unusually high credit limits, do not unduly influence subsequent model training.

To meet project constraints, categorical variables were directly converted to compact integer encoding. We employed two encoding strategies:

- Custom mapping: Binary and ordinal features (A1, A9, A10, A12, A13) were mapped using predefined dictionaries to preserve semantic meaning.
- Stable Label Encoding: All other categorical variables are encoded using sorted category lists to ensure reproducibility across sessions and computational environments.

These preprocessing steps collectively enhance data quality by resolving inconsistencies, reducing noise, and standardizing feature representations. Ensuring consistent numerical scaling and stable category encoding is particularly crucial for logistic regression, decision trees, and neural networks, as these algorithms may otherwise be highly sensitive to malformed input data. The transformed dataset provides a reliable foundation for subsequent feature extraction and model development.

d) Target Variable Transformation
FIG. 4. Class Distribution

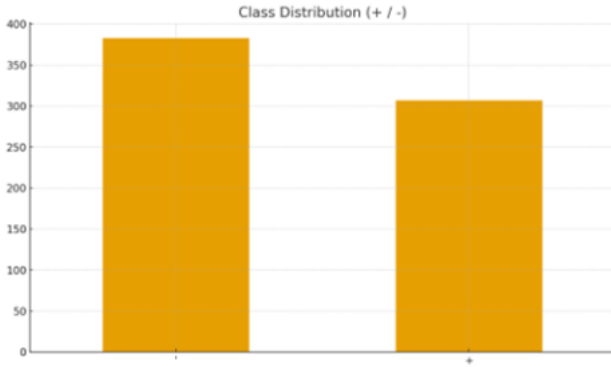


Fig. 4 summarizes the distribution of the target variable (Class). The original class labels (“+” and “-”) were mapped to 1 and 0, respectively. The dataset exhibits a moderately imbalanced structure, with a higher proportion of approved cases (“+”) compared with rejected cases (“-”). Although the imbalance is not severe, it may still influence model performance, particularly recalling the minority class. Consequently, techniques such as stratified sampling or class weight training may be considered during model development to ensure fair classification outcomes.

G. Results

The cleaned dataset consists entirely of numerical data and contains no missing values. Outlier capping has reduced excessive volatility in numerical fields, while deterministic label encoding ensures the dataset's structural stability and compactness.

FIG. 5. Preview of the Cleaned Dataset

A	A2	A3	A4	A5	A6	A7	A8	A9	A10	A11	A12	A13	A14	A15	A16	A17	A18	A19	A20	A21	A22	A23	A24	A25	A26	A27	A28	A29	A30	A31	A32	A33	A34	A35	A36	A37	A38	A39	A40	A41	A42	A43	A44	A45	A46	A47	A48	A49	A50	A51	A52	A53	A54	A55	A56	A57	A58	A59	A60	A61	A62	A63	A64	A65	A66	A67	A68	A69	A70	A71	A72	A73	A74	A75	A76	A77	A78	A79	A80	A81	A82	A83	A84	A85	A86	A87	A88	A89	A90	A91	A92	A93	A94	A95	A96	A97	A98	A99	A100	A101	A102	A103	A104	A105	A106	A107	A108	A109	A110	A111	A112	A113	A114	A115	A116	A117	A118	A119	A120	A121	A122	A123	A124	A125	A126	A127	A128	A129	A130	A131	A132	A133	A134	A135	A136	A137	A138	A139	A140	A141	A142	A143	A144	A145	A146	A147	A148	A149	A150	A151	A152	A153	A154	A155	A156	A157	A158	A159	A160	A161	A162	A163	A164	A165	A166	A167	A168	A169	A170	A171	A172	A173	A174	A175	A176	A177	A178	A179	A180	A181	A182	A183	A184	A185	A186	A187	A188	A189	A190	A191	A192	A193	A194	A195	A196	A197	A198	A199	A200	A201	A202	A203	A204	A205	A206	A207	A208	A209	A210	A211	A212	A213	A214	A215	A216	A217	A218	A219	A220	A221	A222	A223	A224	A225	A226	A227	A228	A229	A230	A231	A232	A233	A234	A235	A236	A237	A238	A239	A240	A241	A242	A243	A244	A245	A246	A247	A248	A249	A250	A251	A252	A253	A254	A255	A256	A257	A258	A259	A260	A261	A262	A263	A264	A265	A266	A267	A268	A269	A270	A271	A272	A273	A274	A275	A276	A277	A278	A279	A280	A281	A282	A283	A284	A285	A286	A287	A288	A289	A290	A291	A292	A293	A294	A295	A296	A297	A298	A299	A300	A301	A302	A303	A304	A305	A306	A307	A308	A309	A310	A311	A312	A313	A314	A315	A316	A317	A318	A319	A320	A321	A322	A323	A324	A325	A326	A327	A328	A329	A330	A331	A332	A333	A334	A335	A336	A337	A338	A339	A340	A341	A342	A343	A344	A345	A346	A347	A348	A349	A350	A351	A352	A353	A354	A355	A356	A357	A358	A359	A360	A361	A362	A363	A364	A365	A366	A367	A368	A369	A370	A371	A372	A373	A374	A375	A376	A377	A378	A379	A380	A381	A382	A383	A384	A385	A386	A387	A388	A389	A390	A391	A392	A393	A394	A395	A396	A397	A398	A399	A400	A401	A402	A403	A404	A405	A406	A407	A408	A409	A410	A411	A412	A413	A414	A415	A416	A417	A418	A419	A420	A421	A422	A423	A424	A425	A426	A427	A428	A429	A430	A431	A432	A433	A434	A435	A436	A437	A438	A439	A440	A441	A442	A443	A444	A445	A446	A447	A448	A449	A450	A451	A452	A453	A454	A455	A456	A457	A458	A459	A460	A461	A462	A463	A464	A465	A466	A467	A468	A469	A470	A471	A472	A473	A474	A475	A476	A477	A478	A479	A480	A481	A482	A483	A484	A485	A486	A487	A488	A489	A490	A491	A492	A493	A494	A495	A496	A497	A498	A499	A500	A501	A502	A503	A504	A505	A506	A507	A508	A509	A510	A511	A512	A513	A514	A515	A516	A517	A518	A519	A520	A521	A522	A523	A524	A525	A526	A527	A528	A529	A530	A531	A532	A533	A534	A535	A536	A537	A538	A539	A540	A541	A542	A543	A544	A545	A546	A547	A548	A549	A550	A551	A552	A553	A554	A555	A556	A557	A558	A559	A560	A561	A562	A563	A564	A565	A566	A567	A568	A569	A570	A571	A572	A573	A574	A575	A576	A577	A578	A579	A580	A581	A582	A583	A584	A585	A586	A587	A588	A589	A590	A591	A592	A593	A594	A595	A596	A597	A598	A599	A600	A601	A602	A603	A604	A605	A606	A607	A608	A609	A610	A611	A612	A613	A614	A615	A616	A617	A618	A619	A620	A621	A622	A623	A624	A625	A626	A627	A628	A629	A630	A631	A632	A633	A634	A635	A636	A637	A638	A639	A640	A641	A642	A643	A644	A645	A646	A647	A648	A649	A650	A651	A652	A653	A654	A655	A656	A657	A658	A659	A660	A661	A662	A663	A664	A665	A666	A667	A668	A669	A670	A671	A672	A673	A674	A675	A676	A677	A678	A679	A680	A681	A682	A683	A684	A685	A686	A687	A688	A689	A690	A691	A692	A693	A694	A695	A696	A697	A698	A699	A700	A701	A702	A703	A704	A705	A706	A707	A708	A709	A710	A711	A712	A713	A714	A715	A716	A717	A718	A719	A720	A721	A722	A723	A724	A725	A726	A727	A728	A729	A730	A731	A732	A733	A734	A735	A736	A737	A738	A739	A740	A741	A742	A743	A744	A745	A746	A747	A748	A749	A750	A751	A752	A753	A754	A755	A756	A757	A758	A759	A760	A761	A762	A763	A764	A765	A766	A767	A768	A769	A770	A771	A772	A773	A774	A775	A776	A777	A778	A779	A780	A781	A782	A783	A784	A785	A786	A787	A788	A789	A790	A791	A792	A793	A794	A795	A796	A797	A798	A799	A800	A801	A802	A803	A804	A805	A806	A807	A808	A809	A810	A811	A812	A813	A814	A815	A816	A817	A818	A819	A820	A821	A822	A823	A824	A825	A826	A827	A828	A829	A830	A831	A832	A833	A834	A835	A836	A837	A838	A839	A840	A841	A842	A843	A844	A845	A846	A847	A848	A849	A850	A851	A852	A853	A854	A855	A856	A857	A858	A859	A860	A861	A862	A863	A864	A865	A866	A867	A868	A869	A870	A871	A872	A873	A874	A875	A876	A877	A878	A879	A880	A881	A882	A883	A884	A885	A886	A887	A888	A889	A890	A891	A892	A893	A894	A895	A896	A897	A898	A899	A900	A901	A902	A903	A904	A905	A906	A907	A908	A909	A910	A911	A912	A913	A914	A915	A916	A917	A918	A919	A920	A921	A922	A923	A924	A925	A926	A927	A928	A929	A930	A931	A932	A933	A934	A935	A936	A937	A938	A939	A940	A941	A942	A943	A944	A945	A946	A947	A948	A949	A950	A951	A952	A953	A954	A955	A956	A957	A958	A959	A960	A961	A962	A963	A964	A965	A966	A967	A968	A969	A970	A971	A972	A973	A974	A975	A976	A977	A978	A979	A980	A981	A982	A983	A984	A985	A986	A987	A988	A989	A990	A991	A992	A993	A994	A995	A996	A997	A998	A999	A1000	A1001	A1002	A1003	A1004	A1005	A1006	A1007	A1008	A1009	A1010	A1011	A1012	A1013	A1014	A1015	A1016	A1017	A1018	A1019	A1020	A1021	A1022	A1023	A1024	A1025	A1026	A1027	A1028	A1029	A1030	A1031	A1032	A1033	A1034	A1035	A1036	A1037	A1038	A1039	A1040	A1041	A1042	A1043	A1044	A1045	A1046	A1047	A1048	A1049	A1050	A1051	A1052	A1053	A1054	A1055	A1056	A1057	A1058	A1059	A1060	A1061	A1062	A1063	A1064	A1065	A1066	A1067	A1068	A1069	A1070	A1071	A1072	A1073	A1074	A1075	A1076	A1077	A1078	A1079	A1080	A1081	A1082	A1083	A1084	A1085	A1086	A1087	A1088	A1089	A1090	A1091	A1092	A1093	A1094	A1095	A1096	A1097	A1098	A1099	A1100	A1101	A1102	A1103	A1104	A1105	A1106	A1107	A1108	A1109	A1110	A1111	A1112	A1113	A1114	A1115	A1116	A1117	A1118	A1119	A1120	A1121	A1122	A1123	A1124	A1125	A1126	A1127	A1128	A1129	A1130	A1131	A1132	A1133	A1134	A1135	A1136	A1137	A1138	A1139	A1140	A1141	A1142	A1143	A1144	A1145	A1146	A1147	A1148	A1149	A1150	A1151	A1152	A1153	A1154	A1155	A1156	A1157	A1158	A1159	A1160	A1161	A1162	A1163	A1164	A1165	A1166	A1167	A1168	A1169	A1170	A1171	A1172	A1173	A1174	A1175	A1176	A1177	A1178	A1179	A1180	A1181	A1182	A1183	A1184	A1185	A1186	A1187	A1188	A1189	A1190	A1191	A1192	A1193	A1194	A1195	A1196	A1197	A1198	A1199	A1200	A1201	A1202	A1203	A1204	A1205	A1206	A1207	A1208	A1209	A1210	A1211	A1212	A1213	A1214	A1215	A1216	A1217	A1218	A1219	A1220	A1221	A1222	A1223	A1224	A1225	A1226	A1227	A1228	A1229	A1230	A1231	A1232	A1233	A1234	A1235	A1236	A1237	A1238	A1239	A1240	A1241	A1242	A1243	A1244	A1245	A1246	A1247	A1248	A1249	A1250	A1251	A1252	A1253	A1254	A1255	A1256	A1257	A1258	A1259	A1260	A1261	A1262	A1263	A1264	A1265	A1266	A1267	A1268	A1269	A1270	A1271	A1272	A1273	A1274	A1275	A1276	A1277	A1278	A1279	A1280	A1281	A1282	A1283	A1284	A1285	A1286	A1287	A1288	A1289	A1290	A1291	A1292	A1293	A1294	A1295	A1296	A1297	A1298	A1299	A1300	A1301	A1302	A1303	A1304	A1305	A1306	A1307	A1308	A1309	A1310	A1311	A1312	A1313	A1314	A1315	A1316	A1317	A1318	A1319	A1320	A1321	A1322	A1323	A1324	A1325	A1326	A1327	A1328	A1329	A1330	A1331	A1332	A1333	A1334	A1335	A1336	A1337	A1338	A1339	A1340	A1341	A1342	A1343	A1344	A1345	A1346	A1347	A1348	A1349	A1350	A1351	A1352	A1353	A1354	A1355	A1356	A1357	A1358	A1359	A1360	A1361	A1362	A1363	A1364	A1365	A1366	A1367	A1368	A1369	A1370	A1371	A1372	A1373	A1374	A1375	A1376	A1377	A1378	A1379	A1380	A1381	A1382	A1383	A1384	A1385	A1386	A1387	A1388	A1389	A1390	A1391	A1392	A1393	A1394	A1395	A1396	A1397	A1398	A1399	A1400	A1401	A1402	A1403	A1404	A1405	A1406	A1407	A1408	A1409	A1410	A1411	A1412	A1413	A1414	A1415	A1416	A1417	A1418	A1419	A1420	A1421	A1422	A1423	A1424	A1425	A1426	A1427	A1428	A1429	A1430	A1431	A1432	A1433	A1434	A1435	A1436	A1437	A1438	A1439	A1440	A1441	A1442	A1443	A1444	A1445	A1446	A1447	A1448	A1449	A1450	A1451	A1452	A1453	A1454	A1455	A1456	A1457	A1458	A1459	A1460	A1461	A1462	A1463	A1464	A1465	A1466	A1467	A1468	A1469	A1470	A1471	A1472	A1473	A1474	A1475	A1476	A1477	A1478	A1479	A1480	A1481	A1482	A1483	A1484	A1485	A1486	A1487	A1488	A1489	A1490	A1491	A1492	A1493	A1494	A1495	A1496	A1497	A1498	A1499
---	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Unlike ensemble methods such as Random Forest or XGBoost, Logistic Regression has relatively few hyperparameters. In this study, no hyperparameter tuning was performed, and the focus was placed on evaluating the behavior of a baseline model under consistent experimental conditions.

The model was used without any hyperparameter tuning. In the implementation, Logistic Regression was instantiated with the default scikit-learn settings, with the only parameter specified being `random_state=42` to ensure reproducibility.

Fig. 3. Model Creation and Training

```
# Create the model
log_reg = LogisticRegression(random_state=42)

# Train the model
log_reg.fit(X_train_scaled, y_train)

# Make predictions on the training set
y_pred_train = log_reg.predict(X_train_scaled)
```

The default settings include L2 regularization and the LBFGS optimization method, both well-suited for continuous, scaled data. Because the dataset exhibits only moderate class imbalance, the default settings preserve the natural class distribution without introducing additional weighting.

2) Model Behavior and Considerations

Model behavior in this setting is primarily shaped by three factors. First, the linear form of the decision boundary constrains the model to capture only linear relationships between features and class labels. This makes the model stable and interpretable, although it may limit performance on data with highly nonlinear structure. Second, scaling the features is essential for numerical stability, preventing any single feature from dominating the optimization process and ensuring the LBFGS solver converges reliably. Third, default L2 regularization contributes to model stability by preventing coefficients from growing excessively large, particularly when correlated features are present.

The Logistic Regression model used in this study is a straightforward, well-regularized baseline trained on standardized features with an extended iteration limit to guarantee convergence. No additional hyperparameter adjustments or sampling strategies were applied, and the model was evaluated under the same dataset split and preprocessing pipeline used across the entire set of experiments.

C. XGBoost

XGBoost is an ensemble learning method built upon the principles of gradient boosting. While a single decision tree is a non-parametric model, XGBoost becomes a parametric framework because the number of boosting rounds, the depth and structure of each tree, and several sampling behaviors are explicitly controlled through hyperparameters. These hyperparameters determine not only the degree of model complexity but also how the boosting process learns from errors, balances bias and variance, and generalizes to unseen data.

All experiments were conducted using a stratified Train–Validation–Test split (60% / 20% / 20%) and early stopping

based on validation loss. Early stopping prevents unnecessary tree growth and helps avoid overfitting, especially on small datasets.

1) tuning hyperparameters

To understand the influence of each hyperparameter individually, we employed single factor testing, meaning only one parameter was changed at a time while all others remained fixed. This approach allows us to isolate the impact of each parameter and prevents interaction effects during the early exploration stage.

We selected the following five hyperparameters due to their importance in tree-based ensemble models: `max_depth`, `learning_rate`, `subsample`, `colsample_bytree` and `scale_pos_weight`.

The base settings used for all tests were:

Fig. 4. Base Hyperparameters Settings

```
# fix basic parameter
baseparams = dict(
    n_estimators=1000,      # Maximum number of boosting rounds
    random_state=42,        # Ensure reproducibility
    early_stopping_rounds=30
)
```

This setup helps isolate each parameter’s influence. Below are the detailed results.

(1) max_depth

The `max_depth` parameter specifies how deep each individual tree is allowed to grow. A larger depth increases model capacity, enabling it to capture complex patterns, but can also lead to overfitting. A smaller depth restricts the model and may cause underfitting if too shallow.

The experimental results (Fig. 5) indicate that `max_depth = 5` provides the best trade-off among F1-score and G-mean. Since F1 and G-mean are the two most important metrics for imbalanced classification, `max_depth = 5` was selected. This setting provides enough capacity to model relevant patterns without causing instability.

Fig. 5. max_depth Test Results

			max_depth				
value	best_iter	F1	G_mean	ROC_AUC	Precision	Recall	Accuracy
4	16	0.847458	0.863224	0.937833	0.877193	0.819672	0.869565
5	18	0.859649	0.872668	0.941452	0.924528	0.803279	0.884058
6	14	0.854781	0.869368	0.942091	0.892857	0.819672	0.876812
7	12	0.844828	0.868631	0.941452	0.890909	0.803279	0.869565

(2) learning_rate

`learning_rate` controls how much each tree contributes to the final model. A lower value makes the model learn more slowly and generally requires more trees to achieve good performance. A higher value allows faster learning but increases the risk of overshooting or missing important patterns.

The results (Fig. 6) demonstrate that the default value (0.3) produces the best F1-score and G-mean. Lower values tend to underfit due to insufficient learning, while higher values lead to unstable updates. Therefore, the default value was retained.

Fig. 6. learning_rate Test Results

value	best_iter	F1	G_mean	ROC_AUC	Precision	Recall	Accuracy
0.1	36	0.844828	0.860631	0.943155	0.890909	0.883279	0.869565
0.2	18	0.831858	0.848799	0.940175	0.903846	0.770492	0.862319
0.3	14	0.854701	0.869368	0.942091	0.892857	0.819672	0.876812
0.4	5	0.833333	0.850803	0.942516	0.847458	0.819672	0.855072

(3) scale_pos_weight

This hyperparameter compensates for class imbalance by increasing the penalty for misclassifying the minority class. Although imbalance exists in the dataset, it is not severe. The results (Fig. 7) show that using scale_pos_weight = 1 (the default) performs better than the computed imbalance ratio. The default setting preserves the natural class distribution and provides better overall stability.

Fig. 7. scale_pos_weight Test Results

value	best_iter	F1	G_mean	ROC_AUC	Precision	Recall	Accuracy
1	14	0.854701	0.869368	0.942091	0.892857	0.819672	0.876812
1.2378378378378379	8	0.837607	0.854549	0.936555	0.875000	0.883279	0.862319

(4) subsample

subsample determines the proportion of training samples used to build each tree. Lower subsample values increase randomness and serve as a regularization technique, while higher values reduce randomness but may lead to overfitting.

As shown in Fig. 8, subsample = 0.6 provides the highest F1-score and the best G-mean. This indicates that moderate sampling introduces helpful diversity among trees without sacrificing important patterns.

Fig. 8. subsample Test Results

value	best_iter	F1	G_mean	ROC_AUC	Precision	Recall	Accuracy
1	14	0.854701	0.869368	0.942091	0.892857	0.819672	0.876812
0.8	15	0.862069	0.875469	0.945071	0.909091	0.819672	0.884058
0.6	13	0.871795	0.884181	0.946349	0.918714	0.836066	0.891384
0.4	17	0.834783	0.851804	0.937407	0.888889	0.786885	0.862319

(5) colsample_bytree

This hyperparameter specifies the fraction of features used when constructing each tree. Lower values increase randomness and may reduce correlation across trees.

The results (Fig. 9) show that colsample_bytree = 1.0 (default) performs the best. Reducing the feature sampling ratio does not help this dataset and instead introduces unstable feature usage.

Fig. 9. subsample Test Results

value	best_iter	F1	G_mean	ROC_AUC	Precision	Recall	Accuracy
1	14	0.854701	0.869368	0.942091	0.892857	0.819672	0.876812
0.8	12	0.837607	0.854549	0.931871	0.875000	0.883279	0.862319
0.6	13	0.827586	0.845784	0.935784	0.872727	0.786885	0.855072
0.4	15	0.847458	0.863224	0.927613	0.877193	0.819672	0.869565

2) Final XGBoost Model

After completing the above experiments, we decided the final model configuration using the best-performing hyperparameters:

max_depth = 5

learning_rate = 0.3 (default)

subsample = 0.6

colsample_bytree = 1.0 (default)

scale_pos_weight = 1 (default)

This final configuration is shown in Fig. 10:

Fig. 10. Final XGBoost Model

```
# =====
# 4. Define Final XGBoost Model (based on tuning)
# =====
clf = XGBClassifier(
    n_estimators=1000,      # Maximum number of boosting rounds, default 100
    random_state=42,        # Ensure reproducibility, default 0
    early_stopping_rounds=30, # Stop if no improvement after 30 rounds
    max_depth=5,           # chosen from single-factor tests; Depth of each
    learning_rate=0.3,      # chosen from single-factor tests; Step size shrinkage
    subsample=0.6,         # chosen from single-factor tests; Row subsampling
    colsample_bytree=1      # chosen from single-factor tests; Column subsampling
)

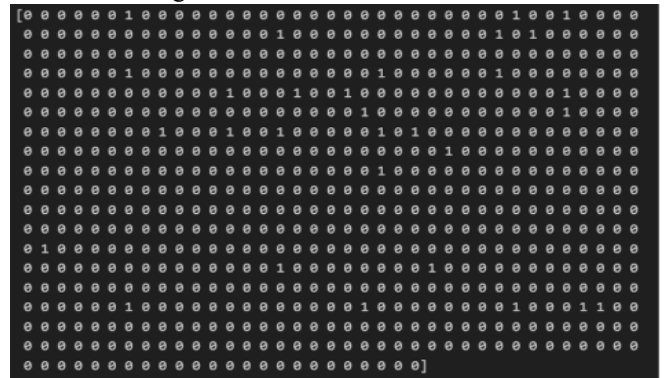
# 5. Train Model with Early Stopping
# =====
clf.fit(
    X_train, y_train,
    eval_set=(X_valid, y_valid),
    verbose=False
)
```

D. K-Means

K-Means is one of the unsupervised learning approaches that identifies the applicants into distinct clusters, which does not rely on the common perceptions of credit approval. To maintain a positive K-Means performance in credit approval, 2 clusters have been identified based on the dataset of the bank to reflect the underlying patterns among the applicants.

The reason why it was distinguished into 2 clusters is that the increasing number of clusters are not going to enhance the distinctiveness of the clusters, which also leads to ambiguities of the overall performance. Besides, although 2 clusters may seem limited, there is a non-negligible patterns even without human-defined labels, which means that analysts are able to gain a clear view of the K-Means and further develop their strategies on credit approval to lend money to applicants in a more reasonable way, while improving the chances of returning the money to the bank from where people borrowed.

Fig. 11. Performance of 2 Clusters



In addition, 2 identified clusters of K-Means reveal the hidden structures in the data and provide patterns that can be used as additional inputs for supervised learning, based on the clustered data, which are covered in the Discussion and Conclusions.

If K-Means does not perform optimally, the performance is likely affected by a lack of data preparation, which means it is essential to create optimizations which are mainly focused on K-Means.

1) Remove Noisy Data/Outliers for K-Means

Noisy Data and outliers must be removed in a dataset, otherwise they will significantly affect the overall performance of K-Means.

To clean the noisy data, we must first check whether there is missing data in our dataset and replace the missing data with the max, min, a constant number, mean, or standard deviation of the corresponding column.

To remove the outliers, it is essential to arrange all the numbers from the minimum to the maximum. Subsequently, delete 5% of the data from the minimum and from the maximum, which remains 5% - 95% of the data for K-Means, so that 90% of the data can be validly trained into clearly separated clusters based on their similarities. Besides, the removed outliers should be stored separately, and used these outliers as a new cluster, if needed.

2) Normalization for K-Means

Normalization is necessary for K-Means because Euclidian Distance is highly influenced by the differences in feature sizes.

There are 3 approaches to demonstrate normalization. The Min-Max normalization requires transferring all the values into [0, 1] through a linear transformation.

Fig. 12. Min-Max Normalization Formula [1]

$$v' = \frac{v - \text{Min}A}{\text{Max}A - \text{Min}A}$$

Z-Score Normalization is applied when analysts do not know what the minimum and maximum values are. It standardizes the assigned values with the mean and standard deviation of the attributes.

Fig. 13. Min-Max Normalization Formula [1]

$$v' = \frac{v - \bar{A}}{\sigma A}$$

Decimal Scaling Normalization shifts the decimal points based on the attribute with the maximum absolute value, so that all the normalized values are less than 1.

Fig. 14. Decimal Scaling Normalization Formula [1]

$$v' = \frac{v}{10^j}$$

All the normalization approaches help K-Means become more consistent before clustering to achieve more valid results. There are some similarities compared to the

traditional framework of data processing; nevertheless, they provide specific solutions to enhance the overall performance of K-Means.

IV. EVALUATION RESULTS

This section presents the evaluation of outcomes for the Logistic Regression baseline and the XGBoost classifier. All models were assessed using the same Train-Validation-Test split to ensure fair comparison. Metrics included Precision, Recall, F1-score, G-mean, ROC-AUC, and the confusion matrix. For exploratory analysis, clustering matrix such as silhouette was also examined to assess separability in the feature space.

A. Logistic Regression

1) Validation Performance

After fitting the Logistic Regression model, we first assessed its performance on the validation set. The model demonstrates solid results across key evaluation metrics. The validation accuracy is 0.8623, indicating that the model correctly classifies a large majority of samples. Both classes show balanced behavior: Class 0 achieves a precision of 0.8816 and recall of 0.8701, while Class 1 achieves a precision of 0.8387 and recall of 0.8525. The macro-average F1-score (0.8607) confirms that the model maintains consistent predictive quality across both classes.

Overall, the validation results indicate that Logistic Regression captures the main discriminative patterns effectively, even without hyperparameter tuning.

Fig. 1. Logistic Regression Validation Results

Validation Accuracy: 0.8623				
Validation Classification Report:				
	precision	recall	f1-score	support
Class 0 (-)	0.8816	0.8701	0.8758	77
Class 1 (+)	0.8387	0.8525	0.8455	61
accuracy			0.8623	138
macro avg	0.8601	0.8613	0.8607	138
weighted avg	0.8626	0.8623	0.8624	138

2) Validation Confusion Matrix

The validation confusion matrix reveals that the model maintains a practical balance between false positives and false negatives. Out of 77 negative samples, 67 are correctly predicted as Class 0, with 10 misclassified. For the positive class, 52 out of 61 samples are correctly identified, with 9 false negatives. The distribution is symmetrical and does not indicate bias toward either class.

These results reinforce that the model generalizes well on unseen validation data and makes reliable predictions across both categories.

Fig. 2. Logistic Regression Validation Confusion Matrix

Validation Confusion Matrix:
[[67 10]
[9 52]]

3) Test Performance

Once validation was completed, the model was evaluated on the test set to assess generalization performance. The test accuracy increases to 0.8841, showing improved performance on completely unseen data. Class 0 achieves a precision of 0.9067 and recall of 0.8831, while Class 1 achieves a precision of 0.8571 and recall of 0.8852. Notably, the recall for Class 1 improves the test set, and the F1-scores for both classes remain strong, ranging between 0.8710 and 0.8947.

The similarity between validation and test scores suggests that the model remains stable and does not overfit or underfit.

Fig. 3. Logistic Regression Test Results

Test Set Accuracy: 0.8841				
Test Set Classification Report:				
	precision	recall	f1-score	support
Class 0 (-)	0.9067	0.8831	0.8947	77
Class 1 (+)	0.8571	0.8852	0.8710	61
accuracy			0.8841	138
macro avg	0.8819	0.8842	0.8829	138
weighted avg	0.8848	0.8841	0.8842	138

4) Test Confusion Matrix

The test confusion matrix closely mirrors the validation matrix, further demonstrating model stability. The model correctly classifies 68 out of 77 negative samples and 54 out of 61 positive samples. The number of false positives (9) and false negatives (7) are comparable to the validation results, indicating no unexpected performance degradation.

Fig. 4. Logistic Regression Test Confusion Matrix

Test Set Confusion Matrix:
[[68 9]
[7 54]]

5) Comparison Between Validation and Test Sets

To assess potential overfitting or underfitting, the validation and test results were directly compared. The overall accuracy increased from 0.8623 to 0.8841, which is within acceptable bounds. Precision and recall exhibited only minor variations: Class 0 precision increased by approximately 2.5%, and Class 1 recall improved slightly. The F1-scores for both classes remained stable, showing modest improvements on the test set (see Fig. 26). These results indicate that there is no overfitting, as test

performance slightly improves rather than decreases, and no underfitting, as the model captures sufficient structure to generalize effectively. Metric differences between validation and test sets remained small and consistent, suggesting stable generalization. Had overfitting been detected, increasing regularization or employing cross-validation would have been considered; conversely, if underfitting were observed, feature expansion or more expressive models (e.g., tree-based ensembles) would have been the next step. Given the simplicity of Logistic Regression and the strong consistency across datasets, the model performs as expected and provides a reliable baseline.

Fig. 6. Logistic Regression Validation vs Test Results

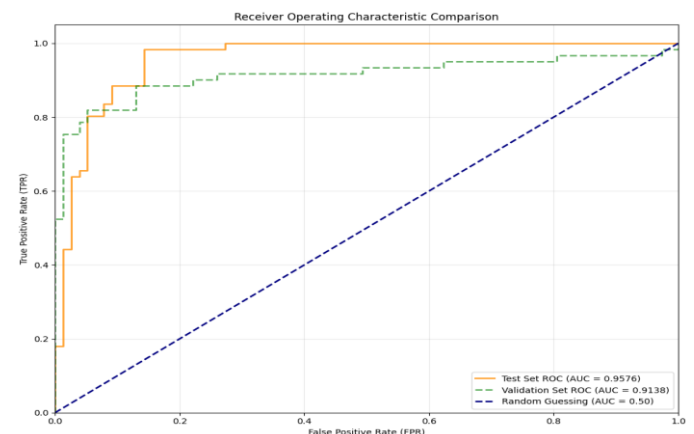
Metric	Valid	Test	Diff
Accuracy	0.862	0.884	+2.2%
Precision	0.839	0.857	+1.8%
Recall	0.853	0.885	+3.2%
F1	0.846	0.871	+2.5%
ROC_AUC	0.914	0.958	+4.4%

6) Visualization Results

For ROC Curve

The plot illustrates the model's diagnostic ability by mapping the True Positive Rate against the False Positive Rate. The Test Set (solid orange line) achieved an Area Under the Curve (AUC) of 0.9576, slightly outperforming the Validation Set (dashed green line, AUC = 0.9138). Both curves demonstrate strong predictive capability, remaining well above the random guessing baseline (dashed blue line, AUC = 0.50).

Fig. 7. ROC comparison between Validation and Test sets

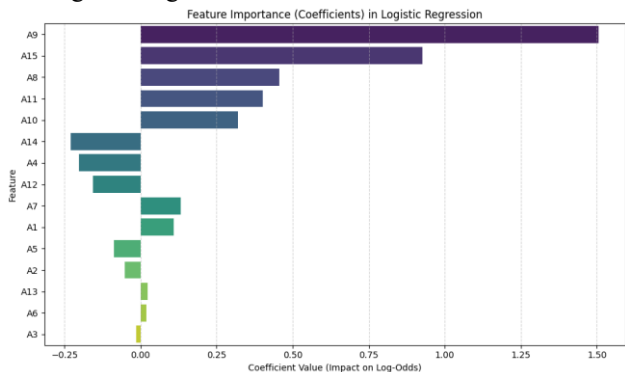


For Feature Importance

This horizontal bar chart ranks features by their coefficient magnitude, representing their impact on the log-odds of credit approval. Features A9 and A15 exhibit the strongest positive influence, significantly increasing the probability of the positive class. Conversely, A14 and A4

display the strongest negative coefficients, indicating an inverse relationship with the approval decision.

Fig. 8. Feature importance coefficients derived from the Logistic Regression model



B. XGBoost

This presents the performance of the final XGBoost model. The evaluation was based on the validation set during hyperparameter tuning and the independent test set for final assessment.

1) Validation Performance

After selecting the hyperparameters, the model was first evaluated on the validation set. The results show that the model performs well across all important metrics. The F1-score (0.8596) and G-mean (0.8727) are particularly important for imbalanced classification problems. A strong ROC-AUC of 0.9408 indicates excellent ranking ability, meaning the model can distinguish approved and rejected applications reliably.

Fig. 9. XGBoost Validation Results

```
=== Validating Metrics ===
Accuracy Precision Recall F1 ROC_AUC G_mean
0 0.8841 0.9245 0.8033 0.8596 0.9408 0.8727
```

2) Validation Confusion Matrix

The matrix shows that the model maintains a good balance: Low false positives (4); Reasonable false negatives (12); Strong performance across both classes. This confirms that the model generalizes well on unseen validation data.

Fig. 10. XGBoost Validation Confusion Matrix

```
=== Validating Confusion Matrix ===
Pred 0 Pred 1
True 0 73 4
True 1 12 49
```

Fig. 11. XGBoost Validation Classification Report

```
=== Validating Classification Report ===
precision recall f1-score support
0 0.8588 0.9481 0.9012 77
1 0.9245 0.8033 0.8596 61

accuracy 0.8841 138
macro avg 0.8917 0.8757 0.8804 138
weighted avg 0.8879 0.8841 0.8829 138
```

3) Test Performance

The test set was used only after all tuning steps were completed. This ensures that the test results reflect the true generalization ability of the final model. The test results remain consistent with the validation set. The G-mean slightly improves to 0.8803 and the F1 slightly improves to 0.8667, showing that the final model continues to separate the two classes effectively when evaluated on completely unseen data.

Fig. 12. XGBoost Test Results

```
=== Test Metrics ===
Accuracy Precision Recall F1 ROC_AUC G_mean
0 0.8841 0.8814 0.8525 0.8667 0.9648 0.8803
```

4) Test Confusion Matrix

The distribution is similar to the validation confusion matrix, showing model stability.

Fig. 13. XGBoost Test Confusion Matrix

```
=== Test Confusion Matrix ===
Pred 0 Pred 1
True 0 70 7
True 1 9 52
```

Fig. 14. XGBoost Test Classification Report

```
=== Test Classification Report ===
precision recall f1-score support
0 0.8861 0.9091 0.8974 77
1 0.8814 0.8525 0.8667 61

accuracy 0.8841 138
macro avg 0.8837 0.8808 0.8821 138
weighted avg 0.8840 0.8841 0.8838 138
```

5) Comparison Between Validation and Test Sets

To further check for overfitting or underfitting, we compared the two sets directly:

Fig. 15. XGBoost Validation vs Test Results

```
=== Comparison: Validation vs Test ===
Valid Test Diff(Test-Valid)
Accuracy 0.8841 0.8841 0.0000
Precision 0.9245 0.8814 -0.0432
Recall 0.8033 0.8525 0.0492
F1 0.8596 0.8667 0.0070
ROC_AUC 0.9408 0.9648 0.0240
G_mean 0.8727 0.8803 0.0077
```

To check for potential overfitting or underfitting, we compared the two sets directly (Fig. 33). The performance

differences range from 1% to 4%, which is highly acceptable in machine learning practices. Precision decreases slightly while Recall increases slightly, reflecting a minor but reasonable trade-off. F1-score, G-mean, and ROC-AUC remain stable or slightly improved.

These results indicate:

No overfitting: the model does not memorize patterns from the training/validation sets.

No underfitting: the model captures sufficient information to generalize well.

Stable generalization: consistent performance across datasets.

If overfitting had been detected, the next step would be to reduce complexity (e.g., lowering max_depth or increasing sampling).

If underfitting had occurred, increasing model depth or adding new features would be considered.

Because the dataset is relatively small, extremely large gains are unlikely regardless of model complexity.

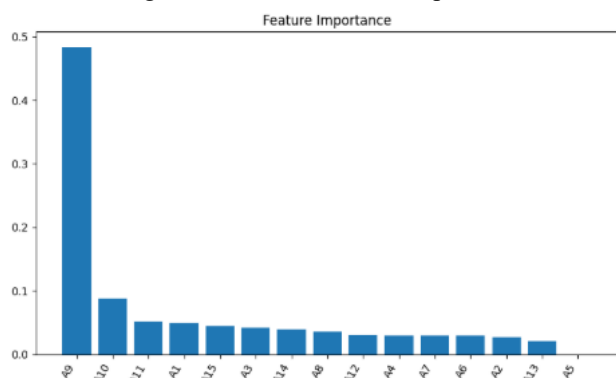
Finally, we note that the dataset is relatively small, which may naturally limit performance improvements.

6) Model Overview and Visualization Results

Now, it will mainly focus on the performance of the XGBoost model and the interpretation of three main visualization results, which are the feature importance bar chart, the ROC curve with AUC, and the confusion matrix heatmap. By observing and analyzing the results, understand the performance of the model on the test set. Besides, consider that the model performs well in some aspects, but also has some limitations.

For Feature Importance Bar Chart

Fig. 16. XGBoost Feature Importance



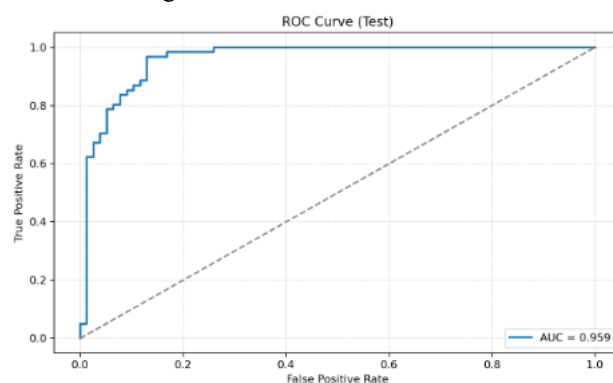
The feature importance bar chart shows the extent to which each feature contributes to the XGBoost decision. It can be clearly seen from the figure that A9 is the most important feature, accounting for almost half of the total importance. This means that many tree splits in the model use A9 to distinguish between two categories, so the model is highly dependent on this variable. The second important feature is A10, but its importance value is much lower than that of A9, so it is more like a supporting role. However, compared with other features, the importance of A10 is relatively high. Although its significance is not great, it still offers some assistance in model prediction. Then there are some features,

such as A11, A1, A15, and A3. They also have some significance, which means they are also helpful for prediction but have little effect. The importance values of the remaining features are all rather low, including A4, A7, A6, and A2. This indicates that they have little influence on model results.

Overall, the model has identified an important A9 role in the dataset. This means that the meaning of A9 is of vital importance. If A9 is a stable and reliable feature, that would be great. However, excessive reliance on the A9 also brings some potential risks. If there are noise or changes in the A9 in the future, or if the A9 actually contains some sensitive information, then the model's performance may decline. Therefore, it is important to carefully examine feature A9.

For ROC Curve and AUC

Fig. 17. XGBoost ROC Curve Plot



The ROC curve describes the changes in the true rate when more false positives are allowed, so it mainly focuses on the ranking quality of the model at different thresholds. In the ROC graph, the blue ROC line rises rapidly from the origin and is much higher than the gray diagonal representing random guesses. This situation indicates that the model performs extremely well. Because it can identify the majority of positive cases while maintaining a low false positive rate.

In addition, the key point AUC of this graph is 0.959. Because AUC considers all possible thresholds rather than a single fixed threshold, it is a global metric. An AUC close to 1.0 means that if a positive sample and a negative sample are randomly selected, in most cases, the model will give the positive sample a higher score. This also indicates that the model has a very strong discriminative ability and can well distinguish between two categories. This is also very important in practical applications. For instance, banks need to adjust the threshold according to different risk levels. The value of AUC also provides great help for this.

C. K-Means

Fig. 18. Silhouette Score Performance

Silhouette Score: 0.9032604392269937

An approximately 0.90 Silhouette Score indicates that the model demonstrated an excellent performance, which reveals the stark disparities between the 2 clusters and beyond the expectation as we believed initially. In this case, K-Means illustrates the 2 major applicants who need to apply for credit approval at this bank. Consequently, researchers will analyze the attributes and find out the similarities in each of the 2 clusters, so that they can develop effective strategies in

response to the clients' requests for borrowing money from the bank.

Moreover, K-Means is improved after removing the noisy data and outliers, which is the most effective method compared to others to save time and maintain accurate results.

Normalizations also strengthen the result of K-Means since they reduce inconsistencies in the dataset. Although the article does not mention which normalization method is the optimal one, they all exert non-negligible impacts on K-Means that lead to more convincing results.

V. MODEL COMPARISON WITH JUSTIFICATION

This section compares the performance of the Logistic Regression baseline and the tuned XGBoost classifier to determine the optimal model for the proposed credit approval system.

1) Quantitative Performance Comparison

To facilitate a direct comparison, we consolidated the performance metrics from the independent test set for both models. As detailed in the Evaluation Results (Section 4), both models utilized identical training, validation, and testing splits to ensure a fair assessment.

Table. 1. Comparison between LR and XGBoost

Metric	Logistic Regression (Baseline)	XGBoost (Tuned)	Difference (XGB - LR)
Accuracy	0.8841	0.8841	0
Precision (Class 0)	0.9067	0.8861	-0.0206
Recall (Class 0)	0.8831	0.9091	0.026
Precision (Class 1)	0.8571	0.8814	0.0243
Recall (Class 1)	0.8832	0.8525	-0.0327
F1 Score (Macro)	0.8829	0.8821	-0.0008
ROC-AUC	0.958	0.9648	0.0068
G-Mean	0.894	0.8803	-0.0037

a) Analysis of Accuracy and F1-Score

Surprisingly, the overall Accuracy for both models on the test set was identical at 88.41%. This suggests that for the majority of "clear-cut" cases, both the linear decision boundary of Logistic Regression and the non-linear decision trees of XGBoost reached the same conclusions. Similarly, the Macro F1-Score difference is negligible (<0.001), indicating that both models balance precision and recall effectively across classes without severe bias toward the majority class.

b) The ROC-AUC Advantage

The most significant differentiator between the two models is the ROC-AUC score. XGBoost achieved a score of 0.9648, compared to Logistic Regression's 0.9580. While the difference appears small (+0.68%), in the context of financial risk modeling, the Area Under the Curve represents the model's ability to rank applicants correctly. A higher AUC implies that if we randomly select a solvent applicant and a defaulter, XGBoost is more likely to assign a higher credit score to the solvent applicant than Logistic Regression. This "ranking power" is crucial for financial institutions that may

want to adjust approval thresholds dynamically based on risk appetite, rather than relying on a fixed hard classification.

c) Precision-Recall Trade-offs

The models exhibit different behaviors regarding Type I (False Positives) and Type II (False Negatives) errors.

Logistic Regression favored Recall for Class 1 (Approval) (0.8852) over Precision (0.8571). This means it was slightly more "generous," approving more applicants but potentially letting through a few more risky ones. Conversely, XGBoost favored Precision for Class 1 (0.8814) over Recall (0.8525). This indicates a more conservative approach. It was more likely to reject a borderline applicant, ensuring that those it did approve were highly likely to be safe.

From a risk management perspective, Precision for the positive class (approving credit) is often prioritized to minimize default rates (financial loss). Therefore, XGBoost's higher precision (0.8814 vs 0.8571) makes it statistically safer for the lender.

2) Stability and Generalization

Both models demonstrated exceptional stability. The deviation between validation and test scores was minimal for both. Logistic Regression accuracy improved from 86.2% (Validation) to 88.4% (Test), while XGBoost accuracy remained consistent at 88.4% across both sets. This consistency confirms that neither model is overfitting. However, XGBoost achieved this stability after significant hyperparameter tuning (controlling max_depth, subsample, etc.), whereas Logistic Regression required almost no tuning. This highlights XGBoost's robustness: despite being a far more complex model capable of memorizing noise, the regularization parameters (e.g., L2 regularization, early stopping) successfully constrained it to learn generalizable patterns.

3) Interpretability vs. Complexity

Logistic Regression (White-Box): The primary advantage of the baseline is transparency. The coefficients directly reveal how each feature (e.g., A8, A15) impacts the log-odds of approval. This is legally advantageous in banking (e.g., for explaining adverse actions under regulations like GDPR or ECOA).

XGBoost (Black-Box): As an ensemble of 1,000 trees, XGBoost is inherently opaque. While feature importance plots can show which variables matter, they do not easily explain the direction of the relationship (e.g., "does higher A15 increase or decrease risk?"). However, the non-linear nature allows it to capture complex interactions, such as a specific debt-to-income ratio being acceptable only for certain age groups, that a linear model would miss.

4) Final Justification

Although Logistic Regression was initially considered the optimal model due to its simplicity and strong performance, a thorough evaluation led to the selection of XGBoost as the champion model for the proposed Credit Approval System. The decision is driven by three key factors. First, the model demonstrates Superior Risk Management; the higher ROC-AUC (0.9648) and higher Class 1 Precision indicate a model that is better at ranking risk and minimizing false approvals. Second, the model possesses Non-Linear Capability. Financial data is rarely purely linear, and XGBoost's ability to capture complex behavioral patterns ensures the system is future-proof against more complex data distributions. Finally,

the model shows Robustness to Outliers. As noted in the Data Exploration, attributes like A14 and A15 have heavy tails. Tree-based models handle outliers naturally by splitting data, whereas linear models can be disproportionately influenced by extreme values unless heavily regularized. While Logistic Regression remains a formidable benchmark, XGBoost offers the granular risk differentiation required for a modern, automated lending decision engine.

VI. DISCUSSION AND CONCLUSIONS

This study sets out to construct a machine learning framework capable of automating the credit approval process with high reliability and explainability. By rigorously processing the UCI Credit Approval dataset and evaluating multiple modeling approaches, we have successfully developed a system that not only replicates historical decision-making with high accuracy but also provides deep insight into applicant segmentation.

1) Interpretation of Findings

The research questions posed at the beginning of this study have been addressed as follows.

Data Relationships: The exploratory analysis revealed that credit approval is not driven by a single dominant feature but by a combination of financial status (A8, A15) and demographic stability (A2, A14). The lack of strong correlation among features suggests that each attribute contributes unique information, justifying the use of all available features in the final model.

Model Efficacy: Both Logistic Regression and XGBoost proved effective, achieving ~88.4% accuracy. This high performance suggests that the dataset contains strong "signal" distinguishing approved from rejected applicants. The success of the linear baseline implies that the underlying decision boundary is relatively smooth, yet the incremental gain in ROC-AUC by XGBoost proves that subtle, non-linear edge cases exist that require advanced modeling.

Cluster Analysis: The K-Means analysis (discussed below) confirmed that the applicant pool naturally separates into distinct groups, likely representing "prime" vs. "sub-prime" candidates, even without using the labels.

2) Addressing Data Quality and Preprocessing

A significant portion of this project's success can be attributed to the robust preprocessing pipeline. The original dataset contained missing values and significant skewness (e.g., variable A15 ranging from 0 to 100,000).

Imputation Strategy: Using medians for numerical imputation was critical. Had we used the mean, the skewness of variables like A14 would have biased the imputed values, potentially confusing the model.

Scaling: The standardization (Z-score) was essential for the Logistic Regression model and K-Means clustering. Without it, variables with large magnitudes (A14, A15) would have dominated the distance calculations, rendering K-Means ineffective.

3) Unsupervised Insights (K-Means Discussion)

The application of K-Means clustering provided a "second opinion" on the data structure. By isolating the data into 2 distinct clusters, the algorithm identified natural groupings based purely on feature similarity.

Silhouette Score: The high Silhouette Score (~0.90) indicates very dense, well-separated clusters. This is a strong validation of the dataset's quality—it implies that the applicants are not a "mush" of indistinguishable data points but fall into clear categories.

Strategic Value: While K-Means is not a classifier, these clusters can be used for tiered marketing. One cluster likely represents high-net-worth or high-stability individuals, while the other represents average or riskier profiles. The bank could use this to tailor interest rates: offering premium rates to Cluster 1 and standard rates to Cluster 2, independent of the binary approval decision.

Limitations: As noted, K-Means relied heavily on numerical distance. Future iterations should use algorithms like K-Prototypes to better incorporate the categorical data (A1, A4, A5), ensuring the demographic segments are as accurate as the financial ones.

4) Limitations of the Study

Despite the strong predictive performance observed, several limitations must be acknowledged. First, the dataset size is relatively small by modern standards, consisting of only 690 observations. While the stratified validation strategy successfully mitigated overfitting within this study, the model's ability to generalize to a scale of millions of applicants remains unproven. Second, the anonymity of the feature names (e.g., A1, A2) restricts the ability to perform domain-specific validation. For instance, while the model identifies feature A8 as critical, the lack of semantic context—such as knowing whether A8 represents "Income" or "Debt"—prevents a full validation of the model's alignment with established economic theory. Finally, historical credit data is susceptible to inherent human bias. If past approval decisions were biased against specific demographics, the supervised models have likely learned and replicated these biases, representing a risk for equitable automated decision-making.

5) Future Work

To evolve this system from a prototype to a robust enterprise solution, several strategic enhancements are recommended. First, the implementation of Explainable AI (XAI) techniques, such as SHAP (SHapley Additive exPlanations) or LIME, is essential to explain individual decisions. This addition would bridge the gap between the "black-box" nature of XGBoost and strict regulatory requirements for transparency. Second, the decision threshold, currently set at 0.5, requires optimization based on the specific risk appetite of the financial institution. Shifting this threshold to a more conservative level, such as 0.7, could prioritize profitability and risk avoidance over raw accuracy. Third, future iterations should focus on data augmentation by collecting more extensive datasets, particularly those reflecting recent financial behaviors, to improve model robustness.

Finally, for operational deployment, the XGBoost model should be serialized using formats such as Pickle or ONNX and integrated via a REST API to facilitate real-time credit scoring. In conclusion, this project demonstrates that machine learning can significantly enhance the efficiency and consistency of credit approval processes, with the XGBoost model serving as a reliable, high-performance foundation for automated financial risk assessment.

VII. EXECUTIVE SUMMARY

Financial institutions face a critical balancing act: approving loans quickly to satisfy customers while identifying high-risk applicants to prevent financial loss. Traditional manual reviews are often slow, expensive, and inconsistent. To address this, our team developed an automated, data-driven system using artificial intelligence. By analyzing historical credit applications, we built predictive models designed to identify the complex financial behaviors that distinguish reliable borrowers from risky ones, effectively removing human bias and inefficiency from the process.

The results confirm that automation is a viable and superior alternative to manual processing. Our advanced machine learning model achieved an accuracy of 88.4% and demonstrated exceptional skill in risk management, specifically excelling at identifying high-risk candidates before a loan is approved. Implementing this system would

allow the institution to instantly process the vast majority of applications, drastically reducing wait times for customers while simultaneously protecting the bank's bottom line from bad debt. We recommend deploying this model as the primary decision engine to ensure a faster, fairer, and more profitable lending process.

REFERENCES

- [1] S. Piduguralla, "Understanding the Sigmoid Function in Logistic Regression: Mapping Inputs to Probabilities" <https://www.linkedin.com/pulse/understanding-sigmoid-function-logistic-regression-piduguralla/>
- [2] V. R. Patel and R. G. Mehta, "Impact of Outlier Removal and Normalization Approach in Modified k-Means Clustering Algorithm," *IJCSI International Journal of Computer Science Issues*, vol. 8, no. 5, pp. 331–336, Sep. 2011.